

Visual Inspection System for Quadruped Robots Based on Large Language Models With a Human-in-the-Loop Mechanism

Muhamad Rizano Alamsyah

Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
ershanoano@email.com

Muhtadin, S.T., M.T.

Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
muhtadin@its.ac.id

Ir. Hany Boedinugroho, M.T.

Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
hanny@its.ac.id

Abstract—Although quadruped robots offer superior mobility for inspections in unstructured terrains, their application is hindered by high cognitive load from manual teleoperation and the rigidity of hard-coded automation. This research aims to integrate a Large Multimodal Model (LMM) agent to automate navigation and visual analysis, while designing a Human-in-the-Loop (HITL) architecture to ensure system adaptability mid-mission. The system development utilizes the Model Context Protocol (MCP) and webhooks to connect natural language instructions with ROS Noetic navigation on a Jueying Lite3 robot. Instructions are broken down into sequential sub-tasks requiring operator validation at each coordinate. At every inspection point, the robot captures an image of the object and compares it against Standard Operating Procedure (SOP) documents to assess condition compliance. Evaluations were conducted through isolated component testing (tool calling, image analysis, plan generation), end-to-end system testing, and operator workload measurement using the NASA-TLX questionnaire. In isolated testing, Gemini 3.1 Pro achieved a tool calling success rate of 98.61%, visual analysis accuracy of 100%, and plan route efficiency of 100%. This advantage carried over to end-to-end testing with a visual analysis accuracy of 93.33% and 100% route efficiency. The system successfully accommodated 100% of spontaneous route interventions from operators and reduced the total NASA-TLX workload score from 54.13 (manual control) to 18.40. The LMM architecture coupled with the HITL mechanism proves highly effective in eliminating hard-coding and significantly alleviating operator cognitive load in autonomous inspections. However, an evaluation of a local model (Qwen 3.5 72B) revealed that local deployment is not yet fully viable for complex missions due to spatial reasoning limitations and computational latency bottlenecks.

Index Terms—quadruped robot, large language model, agentic AI, human-in-the-loop, model context protocol, ROS, autonomous inspection

I. INTRODUCTION

The rapid growth of autonomous mobile robotics—projected to reach USD 9.56 billion by 2030 [1]—reflects increasing demand for automation in high-risk industrial environments. Quadruped robots have emerged as particularly capable platforms for such environments due to their superior terrain adaptability compared to wheeled alternatives [2]. However, despite this mobility advantage,

the deployment of quadruped robots for industrial inspection remains operationally challenging.

Current inspection paradigms fall into two extremes. *Teleoperation* reduces operator risk but imposes continuous cognitive demand, operators must simultaneously interpret raw telemetry and visual feeds while issuing motor commands [3]. *Fully autonomous* approaches address this through deterministic navigation to pre-programmed, as demonstrated by Darmawan [4]. While reducing navigation effort, these systems remain inflexible and typically focus solely on physical movement without automated visual analysis capabilities—meaning any change in inspection scope or criteria requires code-level reprogramming.

Initial attempts at integrating Large Language Models (LLMs) for robot control, such as that proposed by Mahendra [5], offer natural language flexibility but generate and execute complete action plans non-interactively. This “batch-execution” approach provides no mechanism for operator validation or intervention mid-mission, which is critical in dynamic industrial settings where conditions may change unexpectedly [6].

This paper addresses these limitations by proposing a system that is not only autonomous in navigation but also adaptive and responsive to human-robot interaction dynamics [7]. The main contributions of this work are:

- 1) A distributed agentic architecture connecting a multimodal LLM to a ROS-based robot controller via the Model Context Protocol (MCP), enabling semantic command interpretation, autonomous navigation, and automated visual analysis.
- 2) A stepwise Human-in-the-Loop (HITL) mechanism using webhook callbacks that pauses execution after each robot action, awaiting operator validation before proceeding.
- 3) A Agentic Retrieval-Augmented Generation pipeline that grounds LLM decisions in structured inspection data (waypoint coordinates and Standard Operating Procedure documents) retrieved on demand.

II. RELATED WORK

This research builds upon and addresses the limitations of two primary antecedent systems in quadruped robotics for industrial inspection.

A. Deterministic Autonomous Inspection

Darmawan [4] developed an autonomous monitoring system using a quadruped robot to estimate the position of overheating components in substations. The system successfully integrated thermal cameras with YOLOv8s object detection and utilized FastLIO2 for 3D SLAM mapping alongside a combination of Pure Pursuit and Braitenberg algorithms for autonomous navigation. While highly reliable in execution, the system’s navigation was fundamentally deterministic. The robot could only navigate to explicitly hard-coded coordinates, offering no flexibility to adapt to dynamic scenarios or semantic human instructions without code-level reprogramming by technical personnel. Furthermore, the reliance on manual telemetry monitoring continued to impose a high cognitive load on operators.

B. LLM-Based Movement Planning

To introduce semantic flexibility, Mahendra [5] implemented a natural language interaction system using Google Gemini to autonomously generate movement plans for the Jueying Lite3 quadruped robot. The LLM parsed user commands into a complete JSON sequence of navigation actions which were then executed by the ROS Navigation Stack. However, the system utilized a “batch-execution” paradigm: the entire multi-waypoint plan was generated upfront and executed sequentially without interruption. This architecture lacked a step-by-step validation mechanism, stripping operators of the ability to intervene, modify the plan mid-mission, or verify intermediate inspection results before the robot proceeded to the next target.

C. Proposed Contribution

To bridge the gap between flexible language understanding and safe, interactive execution, our work introduces an agentic architecture via the Model Context Protocol (MCP). Unlike [5], our system does not generate a static sequence of actions. Instead, the LLM agent iteratively invokes robotic tools and evaluates their outcomes. By architecturally enforcing a step-wise Human-in-the-Loop mechanism, operators can validate or alter the inspection trajectory after every waypoint, while a Agentic Retrieval-Augmented Generation pipeline ensures the LLM’s decisions are grounded in structured operational data.

III. SYSTEM ARCHITECTURE

A. Overview

The system is designed to bridge the communication gap between human operators and a quadruped robot through a natural language interface. Unlike conventional teleoperation that relies on full manual control, this system places a Large Multimodal Model (LMM) as the central logic unit that orchestrates the information flow from user instructions to

physical robot execution. The system adopts a distributed architecture comprising several loosely coupled components, as illustrated in Fig. 1. Communication between components follows explicit protocols to ensure modularity and replaceability.

B. Component Description

1) *Google Gemini API*: Google Gemini API serves as the brain of the inspection system. Gemini processes natural language commands sent by the operator through the Web Client and analyzes them to formulate inspection plans. Using its *function calling* feature, Gemini accesses various tools available on the MCP Server, deciding when to move the robot, when to capture images, or when to retrieve SOP documents from the database. Gemini API also performs visual analysis during inspection: when the robot camera captures an image, it is sent to Gemini alongside SOP instructions, and the model compares the visual condition of the object against the written standards.

2) *Supabase*: Supabase serves as a Backend-as-a-Service (BaaS) platform providing two primary functions. First, it acts as a relational database storing inspection target locations and coordinates, as well as session history containing conversation logs between the operator and Gemini to maintain mission context. Second, it provides cloud storage for SOP documents and every image captured by the robot camera during the inspection process.

3) *Web Dashboard*: The Web Dashboard is a Next.js application serving as the primary operator control station. It communicates with the backend via REST API for chat and data management. Real-time telemetry (robot pose, battery level, occupancy map, and LiDAR scan) is streamed directly from the ROS ecosystem via *rosbridge* WebSocket. Camera video is delivered over WebRTC through *go2rtc*.

4) *MCP Client (LLM Agent)*: The MCP Client is a FastAPI service that acts as the agentic middleware. It holds the system prompt defining the inspection workflow, manages conversation history in Supabase, and implements the agentic loop: forwarding prompts to Gemini, intercepting *function_call* responses, dispatching tool calls to the MCP Server via MCP over HTTP, and injecting results back into the LLM context. The client also manages the system prompt containing personality configuration and operational constraints that Gemini must follow during missions.

5) *MCP Server*: The MCP Server is a FastMCP HTTP server running on the robot’s onboard AI computer. It operates as a local abstraction layer that exposes all robot functionality as a collection of tools. These tools enable the LLM to move the robot by leveraging ROS functionality, search for waypoint coordinates, capture images from the robot camera, retrieve SOP documents, and perform visual analysis. The server also implements a webhook mechanism to send completion status back to the MCP Client after robot movements finish.

6) *ROS System*: ROS manages all communication and navigation tasks on the robot. It processes raw data from the robot’s LiDAR sensor to generate environment maps and

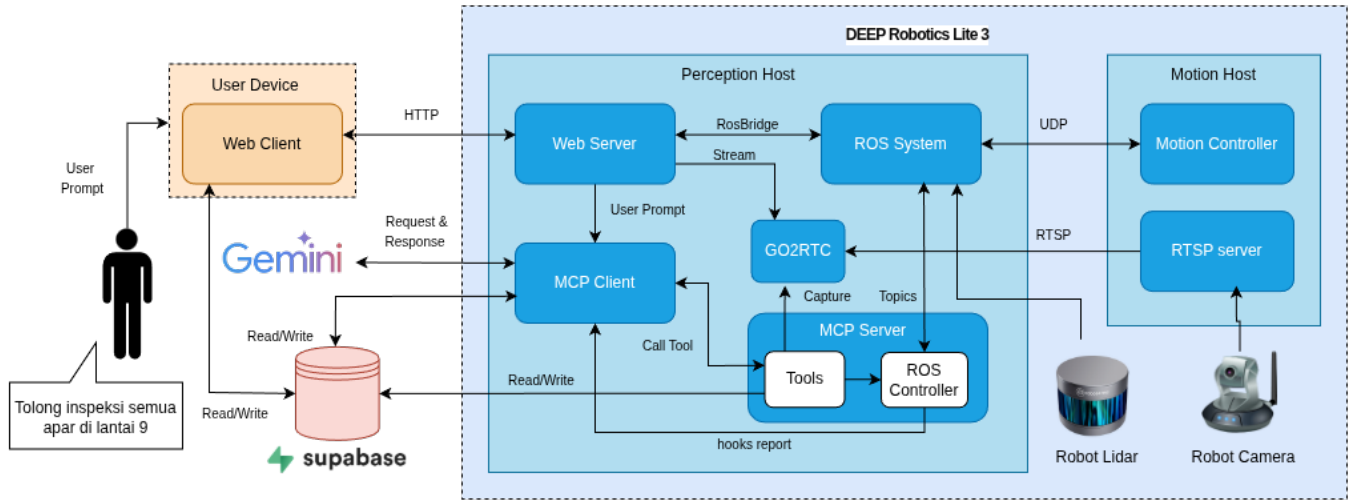


Fig. 1. System architecture diagram showing the distributed components and their communication pathways. The LLM agent (MCP Client) bridges the operator interface and the robot-side MCP Server via the Model Context Protocol.

determine the robot's position accurately during missions. All physical movement coordination is managed through UDP connections to the Motion Host. RosBridge provides Web-Socket connectivity that allows the web interface to access robot telemetry data.

7) *Motion Controller*: The Motion Controller is the lowest-level control unit that interfaces directly with the Jueying Lite3 hardware. It handles dynamic balance and actuator joint rotation (*gait planning*), converting abstract velocity commands from ROS into motor torque signals that physically move the robot.

8) *Video Streaming (GO2RTC and RTSP)*: GO2RTC manages the distribution of visual data from the robot camera to various system modules, receiving data streams via RTSP from the RTSP server connected directly to the camera hardware on the Motion Host. The RTSP server provides raw video data access through the Real-Time Streaming Protocol.

C. Communication Protocols

Table I summarizes the inter-component communication pathways and protocols used throughout the system.

TABLE I
COMMUNICATION PROTOCOLS BETWEEN SYSTEM COMPONENTS

Link	Protocol	Purpose
Web ↔ Backend	REST / HTTP	Chat messages, CRUD
Web ↔ ROS	WebSocket (rosbridge)	Telemetry, maps, LiDAR
Web ↔ Robot camera	WebRTC (go2rtc)	Live video feed
Backend ↔ MCP Server	MCP over HTTP/SSE	Tool invocation
MCP Server ↔ ROS	rospy (topics/actions)	cmd_vel, move_base
Robot → Backend	HTTP POST (webhook)	Navigation completion
Backend ↔ Gemini	Google GenAI SDK	LLM inference
All ↔ Supabase	REST API / SDK	Database & storage
ROS ↔ Motion Host	UDP	Velocity commands
Camera → GO2RTC	RTSP	Video stream

D. Hardware Platform

The robot platform is the DEEPRobotics Jueying Lite3 Pro [14], a 12-DOF quadruped (mass 12.7 kg, 610×370×445 mm standing), augmented with a Leishen C16 16-channel 3D LiDAR [15]. The onboard AI computer is an NVIDIA Jetson Xavier NX, which runs all software components (ROS stack, MCP Server, MCP Client, and web application) as a single-host deployment managed by *systemd* services and Docker Compose. Remote access is provided through Tailscale VPN, eliminating the need for a shared local network.

IV. SYSTEM DESIGN

A. System Workflow

The workflow of the autonomous visual inspection system is divided into three main phases, as illustrated in Fig. 2. All components work synchronously through message exchange and function calling to ensure the inspection mission runs safely and in a directed manner.

1) *Planning Phase*: The first phase focuses on receiving instructions from the operator and formulating a work plan before any physical robot movement. The process begins when the operator inputs a natural language command through the chat panel to inspect specific objects at a given location. The AI Agent receives the message, identifies the need for additional information, and sends requests to the MCP Server to invoke the SOP document search function (*get_sop_file*) and waypoint coordinate data function (*get_object_waypoints*). The MCP Server queries the Supabase database to retrieve object coordinates and relevant SOP content.

After receiving the retrieved data, the AI Agent processes the information to autonomously construct a visual inspection plan based on the priority order of the nearest object locations. The completed plan is sent to the web dashboard for the operator to review. At the end of this phase, the operator reads

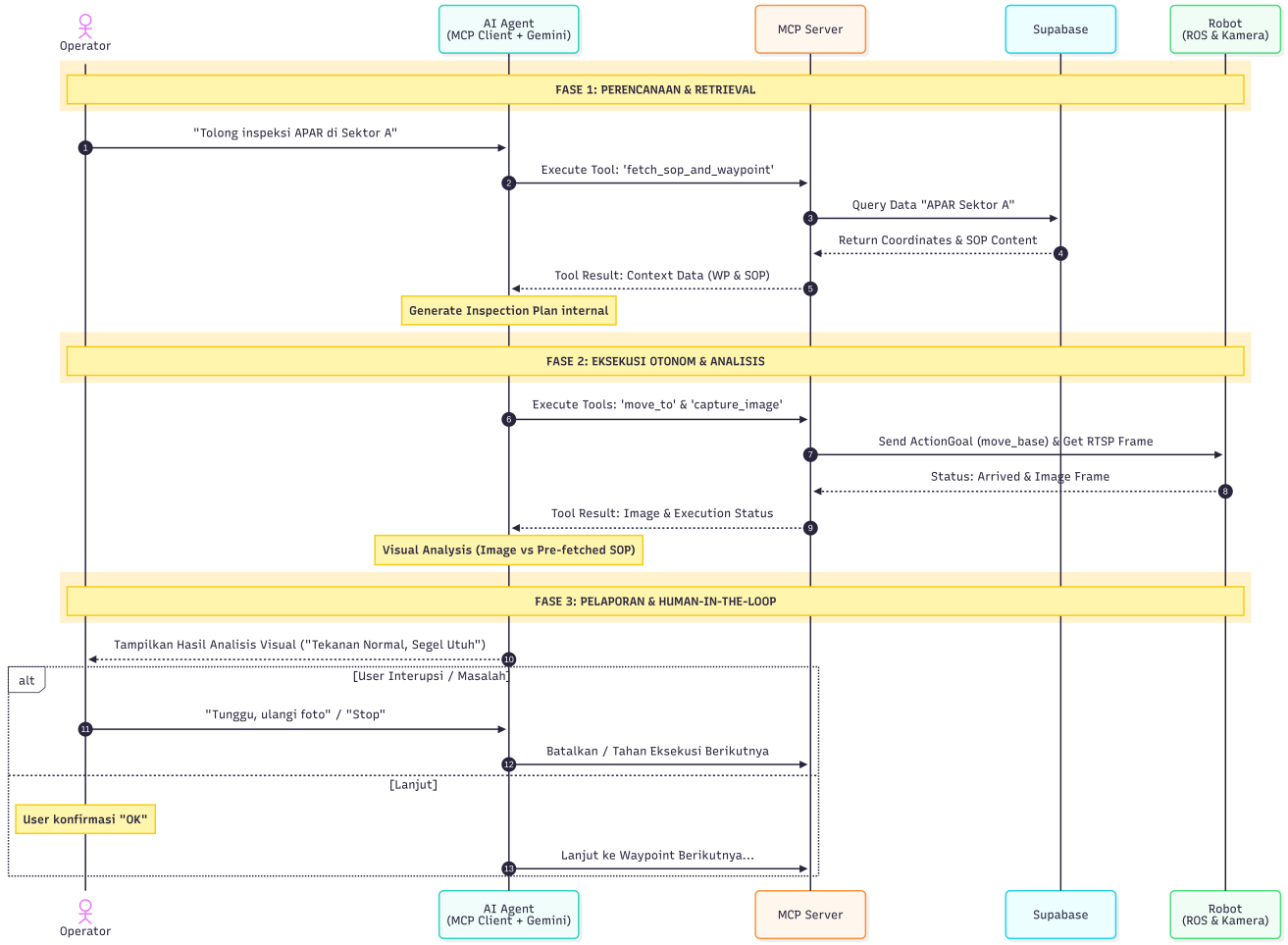


Fig. 2. Sequence diagram showing the three-phase agentic loop: (1) Retrieval & Planning, (2) Autonomous Execution & Analysis, (3) HITL Reporting.

the plan and provides confirmation through chat input as a Human-in-the-Loop control gate before the system enters the next stage.

2) *Execution and Analysis Phase*: Once the plan is approved, the system enters the second phase to physically move the robot and perform visual analysis on target objects. The AI Agent issues an execution command to the MCP Server to invoke the asynchronous navigation tool (`async_navigate_to_waypoint`) to the first target point. The MCP Server translates this command into a ROS message and sends it to the ROS Navigation Stack while simultaneously returning a “running” status to the AI Agent. The robot then moves autonomously toward the target coordinates.

Upon arrival, the MCP Server detects the arrival status and sends a callback report using the webhook mechanism to the AI Agent. After receiving this report, the AI Agent immediately commands the MCP Server to invoke the image capture and visual analysis tool (`capture_and_inspect_image`). The MCP Server accesses the robot camera to take a snapshot via `go2rtc`, then analyzes the image by comparing real-world conditions against

SOP rules using the Gemini multimodal model. After analysis, the processed image is uploaded to Supabase Storage, and the server returns a public image URL along with structured analysis data to the AI Agent.

3) *Reporting Phase*: The final phase involves reporting results to the operator and determining system continuation. The AI Agent displays the visual analysis results along with the field photo to the chat panel for operator review. At this point, the system provides a branching mechanism: if the operator sends an interruption message due to observed anomalies or needs further action, the AI Agent responds immediately and requests the MCP Server to execute tools according to the new operator command (e.g., `async_move` to get closer). Conversely, if conditions are deemed safe and the operator confirms to continue, the AI Agent commands the MCP Server to move the robot to the next waypoint according to the original plan.

B. MCP Tool Definitions

The MCP Server exposes eight tools that collectively span the full capability surface of the robot, as detailed in Table II.

Tool schemas are converted from MCP format to Gemini *FunctionDeclaration* format at runtime by the MCP Client.

TABLE II
MCP TOOLS EXPOSED BY THE ROBOT SERVER

Tool	Description
<code>async_move</code>	Open-loop velocity control. Accepts linear speed (m/s), angular speed (rad/s), and duration (s). Returns “running” immediately.
<code>async_navigate_to_waypoint</code>	Autonomous navigation via <i>move_base</i> Action. Accepts (x, y, yaw). Returns “running” and fires webhook on arrival.
<code>toggle_sit_stand</code>	Sends binary command 0x21010202 to transition robot posture between sitting and standing.
<code>look_up_down</code>	Adjusts body pitch angle (range ± 32767) for a specified duration to tilt the camera view up or down.
<code>get_object_waypoints</code>	Hierarchical DB query: searches objects by keyword across maps $\rightarrow$ locations $\rightarrow$ sub-locations $\rightarrow$ waypoints, returning structured coordinate data.
<code>get_sop_file</code>	Retrieves SOP document metadata and URL links from Supabase. Downloads and text extraction are handled by MCP Client.
<code>capture_and_inspect_image</code>	Captures a frame via go2rtc snapshot, uses Gemini to detect and crop the target object, analyzes against SOP, uploads to Supabase, and returns analysis results.
<code>say_hello</code>	Greeting and conversational response tool.

C. Agentic Loop Processing

The reasoning cycle of the agent is managed within the main processing loop. During execution, the Gemini model may call multiple control functions sequentially within a single conversation turn. However, if the model calls an asynchronous navigation function that returns a “running” status, the client immediately halts the language model’s reasoning loop, saves all state to the database, and directly returns a waiting status response to the operator without blocking the server execution thread. The next reasoning cycle is triggered as a new request after the system receives callback data from the robot system marked with “[ROBOT_FEEDBACK]” text, indicating the robot has arrived at the destination.

This mechanism ensures that long-running physical operations (e.g., 30-second navigation) do not block the server. The webhook-driven architecture decouples LLM inference from robot execution time, enabling the system to remain responsive even during extended autonomous operations.

D. RAG-Based Operational Grounding

Rather than encoding inspection knowledge in the system prompt, the architecture uses tool-time retrieval (Agentic RAG). When Gemini identifies an inspection target, it calls `get_object_waypoints` to fetch spatial coordinates from the database and `get_sop_file` to retrieve the relevant SOP document. This approach ensures that: (1) the LLM always operates on the latest operational data without redeployment; (2) inspection procedures can be updated by operators uploading new SOP files without modifying any code; and (3) hallucination risk is reduced as spatial data is retrieved from a ground-truth database rather than generated.

E. Navigation Stack

The ROS Navigation Stack is configured with a global planner for shortest route computation and a Timed Elastic Band (TEB) local planner [11] for real-time obstacle avoidance. Localization is performed by HDL Localization using NDT scan matching [12] combined with Fast-GICP [13] against a 3D point cloud map built by Faster-LIO (LiDAR-Inertial Odometry). The 3D map is projected to a 2D occupancy grid for use by the Navigation Stack’s costmap.

V. IMPLEMENTATION

A. Deployment

All software components are deployed on the robot’s Jetson Xavier NX. Four *systemd* services manage the core stack: LiDAR driver, navigation (*move_base* + localization), MCP Server (port 8001), and MCP Client/Backend (port 8082). The web dashboard runs in a Docker Compose container (port 3000). Tailscale VPN provides a stable overlay network; the Supabase instance runs on a separate server, mimicking a cloud deployment without public infrastructure requirements.

B. MCP Client Implementation

The MCP Client is built using Python with FastAPI as the core framework, providing high-performance asynchronous HTTP request handling. On Linux, the implementation uses `uvloop` as a replacement for the default `asyncio` event loop to maximize asynchronous data transfer throughput.

1) *Session Management*: Conversation session management stores all inspection activities into the `chat-sessions` and `chat-messages` tables in Supabase. Every new user message, final model text response, internal function call, and tool execution result are stored sequentially. Since Gemini API message structures are complex objects, the MCP Client serializes them into JSONB format for persistent database storage. A status column in the database table serves as a message filter marker, where internal function call logs and raw tool responses are given special flag values so they are not displayed on the user chat panel.

2) *Context Re-injection*: A significant implementation challenge involves maintaining multimodal context across stateless LLM API calls. When historical conversations are loaded from the database, the client implements a context re-injection mechanism: any previously retrieved SOP documents or captured images are re-downloaded from Supabase Storage. For PDF documents and JPEG images, the binary data is directly converted to byte array objects and injected as `Part.from_bytes` into the Gemini input message list. For DOCX documents, the client uses the `python-docx` library to extract all paragraph text first and inject it as plain text message parts, ensuring the language model can read SOP compliance rules in full without losing context.

3) *Schema Conversion*: The MCP Client connects to the MCP Server via HTTP transport using FastMCP. Upon connection, the client requests the list of supported functions. Since FastMCP input parameter schemas are not compatible with Gemini API specifications, the client performs recursive schema conversion by stripping unsupported `title` and `additionalProperties` fields before wrapping them into Gemini `FunctionDeclaration` objects. This prevents function registration failures on the Gemini API.

C. System Prompt Engineering

The system prompt is structured into five key sections that define the agent’s behavior:

- 1) **Role Definition**: Defines the AI as a quadruped inspection robot assistant with explicit responsibilities for planning, tool-based robot control, and transparent operator interaction.
- 2) **Main Workflow**: Instructs the agent to retrieve coordinates and SOP data, formulate a shortest-distance inspection plan, present it to the operator, and *wait for explicit approval*. Each plan point follows a mandatory sub-step sequence: stand → navigate → capture and inspect → report. Points are executed one at a time; between points, the operator is asked whether to continue, modify, or stop.
- 3) **Asynchronous Tool Handling**: Tools prefixed with `async_` return “running” immediately. The agent must halt all tool calls upon receiving this status and wait for the “[ROBOT_FEEDBACK]” signal. Upon receiving arrival feedback, the agent must immediately invoke the image capture tool without waiting for operator instruction.
- 4) **Movement Safety**: When the operator requests the robot to move closer, the agent advances 0.3m per step and records total additional distance. Before proceeding to the next waypoint, a reverse movement equal to the total additional distance plus 0.1m margin is mandatory to prevent navigation drift accumulation.
- 5) **Operational Constraints**: Robot status metadata is distinguished from user instructions. The agent must prioritize physical safety, avoid aggressive movements, never execute plans without operator approval, and always invoke tools rather than merely describing intended actions.

D. MCP Server Implementation

The MCP Server is built using FastMCP in Python, running directly on the robot’s onboard computer. Each registered tool is decorated with special annotations so that input schemas and function descriptions are automatically sent to the client upon connection.

The `capture_and_inspect_image` tool implements a multi-stage pipeline: (1) capture a raw frame from the robot camera via `go2rtc` snapshot protocol; (2) send the image to Gemini with object detection instructions to obtain normalized bounding box coordinates; (3) crop the image using OpenCV

based on detected coordinates with a 20% margin to center the target object; (4) if SOP text is available, perform visual compliance analysis comparing the cropped image against SOP criteria, returning a status of *normal*, *abnormal*, or *inconclusive*; and (5) upload the processed image to Supabase Storage and return the public URL along with structured analysis data.

For asynchronous tools (`async_move` and `async_navigate_to_waypoint`), the server publishes velocity commands or navigation goals to ROS topics and immediately returns a “running” status. Upon goal completion, the ROS action server triggers an HTTP POST webhook callback to the MCP Client, carrying the arrival status message.

E. Web Dashboard

The operator dashboard (Fig. 3) is a single-page Next.js application with TypeScript, utilizing TanStack React Query for state synchronization and caching. The application connects to the backend FastAPI and Supabase database through the official SDK.

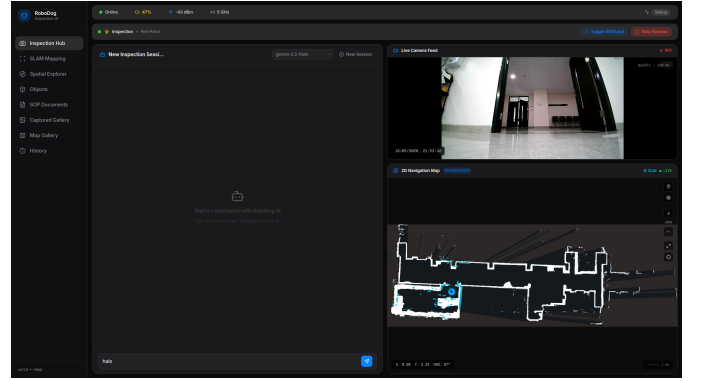


Fig. 3. Web dashboard Inspection Hub: split-screen layout with LLM chat panel (left) and real-time navigation map alongside video feed (right).

The primary *Inspection Hub* page features a split-screen layout. The left panel provides the LLM chat interface with model selection (Gemini 3.1 Pro Preview, Gemini 2.5 Flash, Qwen 3.5 27B). Every operator message automatically includes real-time robot telemetry from RosBridge WebSocket topics: battery level (`/battery_level`), motion status (`/robot_basic_state`), and position coordinates (`/odom`). The chat subscribes to Supabase Realtime channel on the `chat-messages` table, enabling instant rendering of new messages including tool call logs without page reload.

The right panel displays a live video feed via WebRTC (`go2rtc`) and a real-time 2D navigation map. The map processes OccupancyGrid data from the `/map` topic, renders robot position from `/odom`, LiDAR scan points from `/scan`, and the global navigation path from `/move_base/GlobalPlanner/plan` as a dashed yellow line. Operators can set initial pose estimates and navigation goals directly on the map via drag interaction.

An active session bar at the top provides a Sit/Stand toggle button (publishing to `/simple_cmd`) and a Stop Session button that terminates the navigation process, disconnects Web-Socket, and stops the periodic heartbeat signal (0x21040001 at 2 Hz) to prevent the robot from entering a loss-of-control protection mode.

Supporting pages include: *Spatial Explorer* for hierarchical management of maps, locations, sub-locations, and object waypoints; *Object Manager* for defining inspection target categories with keywords and SOP associations; *SOP Manager* for uploading and managing inspection procedure documents; *Capture Gallery* for browsing and downloading images captured during inspection; and *Session History* for reviewing past conversation logs and inspection results.

F. Database Schema

The Supabase PostgreSQL database uses a relational schema with six main tables: `maps` (environment map metadata and YAML config paths), `locations` (hierarchical physical locations with self-referencing `parent_id` and optional arrival coordinates), `objects` (inspection target types with keyword arrays and SOP URL links), `object_waypoints` (navigation coordinates with view position, orientation, and optional camera PTZ parameters, linked to both objects and locations), `chat-sessions` (conversation session metadata), and `chat-messages` (full message history with JSONB content and display filter flags). Supabase Storage provides two folders: `sop` for procedure documents and `captured` for robot camera images, both generating public URLs for display in the chat interface.

VI. EVALUATION

A. Experimental Setup

Experiments were conducted in an indoor laboratory environment replicating an industrial inspection scenario containing three types of inspection objects: fire extinguishers (APAR), electrical outlets, and trash cans. Each object was tested under normal and anomalous conditions (e.g., fire extinguisher lying on the floor, detached hose, damaged plug, scattered trash). All functional tests were performed on three LLM models: Gemini 3.1 Pro Preview (large-scale cloud model with advanced reasoning), Gemini 2.5 Flash (fastest and most cost-efficient cloud model), and Qwen 3.5 27B (locally hosted open-source model for future-proofing). The evaluation comprised four test categories: tool calling, visual analysis, route planning, and full end-to-end integration.

B. Tool Calling and Intent Recognition

The dataset consisted of 72 test cases covering 8 tool types across three difficulty levels: Level 1 (explicit commands naming the tool directly), Level 2 (implicit natural language without explicit tool names), and Level 3 (abstract contextual instructions requiring semantic reasoning). Each tool-level combination was tested 3 times, yielding $8 \times 3 \times 3 = 72$ cases per model.

TABLE III
SUMMARY OF TOOL CALLING EVALUATION

Metric	Gemini 2.5 Flash	Gemini 3.1 Pro	Qwen 3.5 27B
Success Rate	93.06%	98.61%	88.89%
Avg. Latency (s)	2.03	4.11	2.96
Avg. Total Tokens	1,139	1,144	1,587

Gemini 3.1 Pro achieved the highest success rate (98.61%), with all failures concentrated on Level 3. All three models performed perfectly on Levels 1 and 2. The three tools that caused failures at Level 3 were `look_up_down`, `say_hello`, and `toggle_sit_stand`, sharing the characteristic that abstract instructions did not mention robot actions directly. Failure patterns included: (1) models understanding context but choosing to describe actions rather than invoking tools; (2) models failing to associate the instruction with an available tool; and (3) models performing excessive planning before executing the required tool.

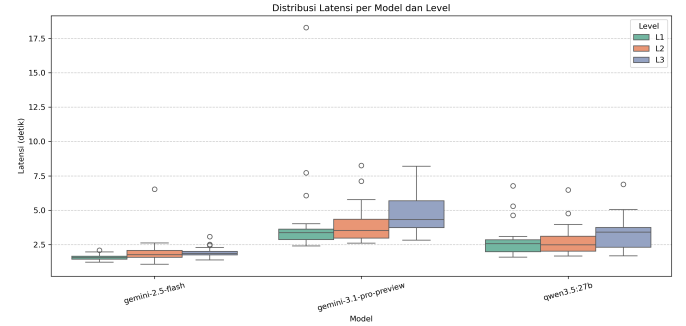


Fig. 4. Latency distribution for tool calling evaluation across three models. Gemini 2.5 Flash demonstrates the lowest median latency, while Gemini 3.1 Pro shows higher latency due to internal reasoning overhead.

As shown in Fig. 4, Gemini 2.5 Flash maintained the shortest response time across all levels. Gemini 3.1 Pro recorded the highest latency due to its deeper internal reasoning mechanism, despite consuming nearly identical token counts to Flash. This confirms that latency is not solely determined by output length but by the complexity of internal reasoning processes.

C. Multimodal Visual Analysis

Visual inspection was evaluated across 3 object types $\times$ 3 conditions $\times$ 3 camera angles = 27 test cases per model. Each condition assessed the model's ability to cross-reference camera captures with retrieved SOP documents.

TABLE IV
SUMMARY OF VISUAL ANALYSIS EVALUATION

Metric	Gemini 2.5 Flash	Gemini 3.1 Pro	Qwen 3.5 27B
Success Rate	85.19%	100.00%	85.19%
Avg. Latency (s)	9.62	19.31	35.29
Avg. Total Tokens	1,980	3,297	5,071

Gemini 3.1 Pro achieved perfect 100% accuracy across all object types and conditions, including the most challenging

scenario: detecting a detached fire extinguisher hose. Gemini 2.5 Flash hallucinated on this scenario, incorrectly stating the hose was properly attached. Qwen 3.5 failed on color detection (misidentifying red extinguisher tubes as yellow due to dominant label sticker pixels) and spatial detail analysis. A common vulnerability was the “detached hose” scenario, which requires fine-grained spatial reasoning about component relationships.

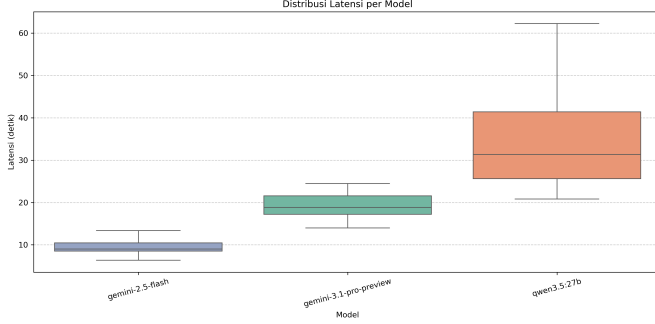


Fig. 5. Latency distribution for visual analysis evaluation. Qwen 3.5 shows significantly higher latency due to local vision encoding overhead on the GPU server.

Fig. 5 reveals a significant trend reversal: Qwen 3.5, which was competitive on text-only tool calling, exhibited the highest latency (~ 35 s median) for multimodal analysis due to the computational burden of local vision encoding. Gemini 2.5 Flash remained the fastest, while Gemini 3.1 Pro’s higher latency correlated with its superior accuracy.

D. Spatial Reasoning and Route Planning

The planning test consisted of 12 cases across four difficulty levels: Easy (1 object), Medium (3 objects), Hard (6 objects), and Impossible (non-existent objects that should be rejected). Evaluation criteria included plan correctness, shortest-route optimality, and rejection of invalid requests.

TABLE V
SUMMARY OF ROUTE PLANNING EVALUATION

Metric	Gemini 2.5 Flash	Gemini 3.1 Pro	Qwen 3.5 27B
Plan Success Rate	91.67%	100.00%	91.67%
Shortest Route Rate	87.50%	100.00%	75.00%
Avg. Latency (s)	25.98	21.85	23.13
Avg. Total Tokens	10,104	5,565	10,396

All three models scored 100% on the Impossible level, demonstrating robust object validation before planning. Gemini 3.1 Pro achieved perfect route optimality across all levels. Both Flash and Qwen 3.5 suffered from a greedy nearest-neighbor approach that selected locally optimal waypoints without considering cumulative distance, resulting in suboptimal total routes on complex scenarios.

A notable trend reversal is shown in Fig. 6: Gemini 2.5 Flash, previously the fastest model, exhibited the highest latency and token consumption on complex planning tasks (spiking to ~ 42 s). This indicates that lightweight models

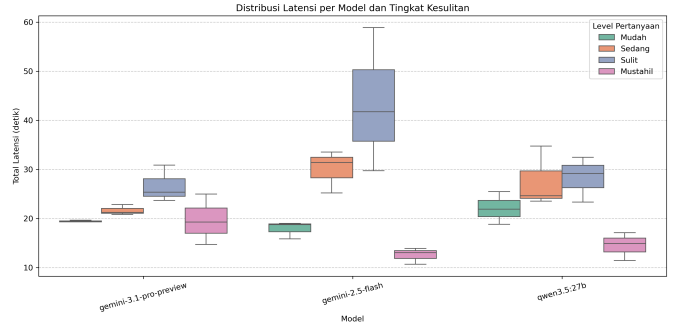


Fig. 6. Latency distribution for route planning evaluation. Gemini 2.5 Flash shows a latency spike on complex tasks due to excessive Chain-of-Thought generation.

require longer Chain-of-Thought scratchpads for high-level spatial reasoning, while Gemini 3.1 Pro’s built-in reasoning engine solved routes more efficiently with fewer output tokens.

E. End-to-End System Performance

The full integration test comprised 9 scenarios across Easy (1 object), Medium (3 objects), and Hard (6 objects) levels, with each level including happy paths, anomaly paths, and HITL intervention paths. Evaluation metrics included plan success, route optimality, visual analysis accuracy, intervention handling, total session time, and token consumption.

TABLE VI
SUMMARY OF END-TO-END SYSTEM EVALUATION

Metric	Gemini 2.5 Flash	Gemini 3.1 Pro	Qwen 3.5 27B
Plan Success	100.00%	100.00%	100.00%
Route Optimality	77.78%	100.00%	66.67%
Visual Analysis	86.67%	93.33%	63.33%
Intervention Success	100.00%	100.00%	100.00%
Avg. Session Time (s)	382.91	367.40	610.79
Avg. Consumed Tokens	153,564	183,440	140,072

All three models achieved 100% plan creation and intervention handling success, proving the MCP architecture’s stability in converting text decisions to physical robot actions and its adaptability to mid-mission operator commands. The HITL mechanism proved highly effective: when operators interrupted missions to inject new targets, the webhook-paused agentic loop accepted the new prompt, queried new coordinates, and resumed navigation without system restart.

Gemini 3.1 Pro maintained 100% route optimality and the highest visual analysis accuracy (93.33%). Its two failures were color misidentification of a fire extinguisher tube due to dominant yellow label sticker pixels—a vulnerability shared across all models and inherent to current LMM architectures. Qwen 3.5 experienced severe visual analysis degradation (63.33%) in E2E scenarios, repeatedly producing false positives on spatially complex anomalies (e.g., failing to detect a fallen extinguisher or a detached hose).

Session time analysis (Fig. 7) revealed a counterintuitive finding: Gemini 2.5 Flash, which was consistently fastest on isolated tests, showed comparable or slower session times

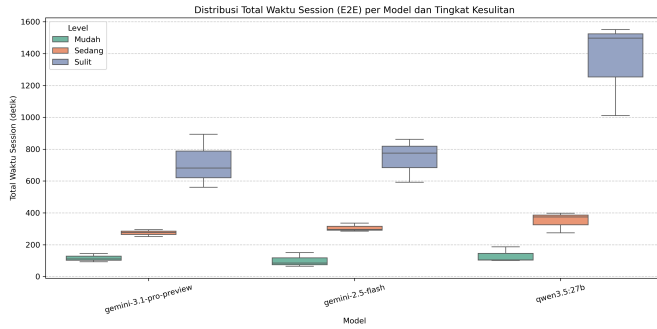


Fig. 7. Session time distribution for E2E evaluation. Qwen 3.5 exhibits significantly higher session times at the Hard level (17–26 minutes) due to accumulated vision encoding overhead.

than Gemini 3.1 Pro at Medium and Hard levels. This is explained by context memory accumulation—as inspection history grows, Flash’s processing rate remains constant (~ 2.5 s per 1,000 tokens), while Pro’s rate actually improves with longer context (dropping from 3.25 to 1.90 s/1,000 tokens), demonstrating superior efficiency scaling. Qwen 3.5 recorded the longest sessions at the Hard level (17–26 minutes), driven by compounding vision encoding latency and context accumulation effects (rising to 4.88 s/1,000 tokens).

F. Operator Workload Assessment (NASA-TLX)

A NASA Task Load Index survey with 10 respondents evaluated perceived operator workload across three conditions: fully manual teleoperation, autonomous navigation with manual visual analysis, and fully autonomous inspection. Results showed a consistent reduction in weighted workload scores: from 54.13 (High) for manual operation, to 34.57 (Medium) for autonomous navigation, to **18.40 (Low)** for fully autonomous inspection. The primary workload reduction sources were Physical Demand (delegating navigation to ROS) and Mental Demand (delegating visual analysis to the LMM). Individual anomalies where workload increased under automation were attributed to system malfunctions eroding trust and the psychological discomfort of transitioning from active controller to passive supervisor.

VII. CONCLUSION

This paper presented an LLM-driven quadruped robot inspection system that addresses the limitations of both hard-coded deterministic navigation and non-interactive LLM planning. By combining the Model Context Protocol for structured tool invocation, webhook-based asynchronous reporting, and a stepwise Human-in-the-Loop validation mechanism, the system enables flexible and safe inspection missions from natural language commands. Experimental results demonstrated that advanced cloud models like Gemini 3.1 Pro can successfully govern complex spatial routing (TSP) and visual anomaly detection, while the architecture itself remains robust enough to integrate local open-source models (e.g., Qwen 3.5) for future-proofing against network outages. The RAG-based grounding

approach ensures that LLM decisions are anchored to authoritative operational data, reducing hallucination risks. Future work will focus on extending the system to outdoor environments, integrating thermal imaging for anomaly detection, and optimizing local vision-language models to reduce inference latency on edge computing devices.

REFERENCES

- [1] Grand View Research, “Autonomous Mobile Robot Market Size, Share & Trends Analysis Report,” 2024. [Online]. Available: <https://www.grandviewresearch.com>
- [2] J. Fan et al., “A review of quadruped robots and environment perception,” *Sensors*, vol. 24, no. 4, 2024.
- [3] D. Rea and C. Anagnostopoulos, “Is teleoperation still relevant for human-robot interaction?” *ACM/IEEE HRI*, 2022.
- [4] Darmawan, “Pengembangan Robot Quadruped untuk Estimasi Posisi Komponen Overheat Pada Gardu Induk Berbasis Kamera Termal,” Tugas Akhir, Institut Teknologi Sepuluh Nopember, 2025.
- [5] V. G. P. A. B. Mahendra, “Interaksi Robot Manusia Berbasis Large Language Model untuk Pembuatan Movement Plan pada Robot Quadruped-legged,” Proposal Tugas Akhir, ITS, 2024.
- [6] W. Huang et al., “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” *ICML*, 2022.
- [7] H. Ravichandar et al., “Recent advances in robot learning from demonstration,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, 2020.
- [8] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv:2204.01691*, 2022.
- [9] D. Driess et al., “PaLM-E: An embodied multimodal language model,” *ICML*, 2023.
- [10] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *NeurIPS*, 2020.
- [11] C. Rösmann et al., “Trajectory modification considering dynamic constraints of autonomous robots,” *ROBOTIK*, 2012.
- [12] P. Biber and W. Straßer, “The normal distributions transform: A new approach to laser scan matching,” *IROS*, 2003.
- [13] K. Koide et al., “Voxelized GICP for fast and accurate 3D point cloud registration,” *ICRA*, 2021.
- [14] DEEPRobotics, *Jueying Lite3 Pro User Manual*, V1.0.7-0, 2024.
- [15] DEEPRobotics, *Jueying Lite3 LiDAR User Manual*, V1.0.7-0, 2025.
- [16] M. Quigley et al., “ROS: an open-source robot operating system,” *ICRA Workshop on Open Source Software*, 2009.